

CSC Problem Set 06 +X Report

Name: Nour Mahmoud

Email: nhmahmou@ncsu.edu

Section: 001

Collaborators: N/A

Summary:

For my +X implementation, I created a competitive dual system where both a human controlled player and an AI controlled agent navigate separate environments simultaneously to collect chips and reach the goal. The game visually displays both robots side-by-side, each with its own environment, and begins with a synchronized countdown timer. I extended the original simulation by modifying the environment setup to support two distinct maps and adding a planning system that dynamically shows the AI's path based on obstacles and key requirements. I also enhanced the user interface by adding interactive controls including Start, Stop, Step, and Reset buttons, as well as a Difficulty dropdown that adjusts the AI's speed in real-time. Finally, I included a visual win display beneath the status panel to clearly indicate the winner, making the simulation more engaging and interactive while showcasing direct competition between human and AI agents.

"+X" Concept:

As a student concentrating in game development, I often find myself instinctively asking, "How can I make this more like a game?" That mindset heavily influenced my +X implementation. Rather than simply extending the planning logic or tweaking heuristics, I wanted to create an experience that felt interactive, competitive, and game-like. This led to the concept of a competitive system, where a human player and an AI agent navigate separate but identical maps, racing to collect chips and reach the goal first. This design gave the player a chance to directly test their own logic, memory, and efficiency against a planning algorithm, turning what was once a passive simulation into a challenge.

To support this idea, I restructured the GUI to include simulation controls: Start, Stop, Step, and Reset buttons, allowing users to pause or progress the simulation at their own pace. I added a Difficulty dropdown that adjusts the AI's speed by modifying the timer delay, giving users a way to challenge themselves further. The implementation also includes a countdown timer visible over the maps and a winner display that appears as soon as one side completes the level. The race begins after the countdown, and the winner is clearly indicated in the sidebar alongside chip counts and inventory. The planning system uses the A* algorithm from PS02, which helps show the user the agent's next steps. The inspiration behind this format came from puzzle and co-op games like [Overcooked](#) [1], where coordination and planning under time constraints are key. The end result isn't just a testing ground for the AI but a mini game, with tension, pacing, and the thrill of beating (or losing to) a robot.

Technical Implementation of +X:

The technical implementation of my +X extension required careful planning and thoughtful modifications across multiple classes in the codebase. At the heart of the feature was the dual-environment setup, where one robot is controlled by a human player using keyboard inputs and the other is the AI agent. To create this system, I had to restructure the environment display, agent logic, and event loop to support simultaneous but independent updates for both robots.

To support keyboard movement for the human-controlled robot, I modified the VisualizeSimulation class by adding a KeyListener that responds to W/A/S/D or arrow keys. On each valid key press, the program checks whether the target tile is accessible using the validPos() method from the Environment class and then updates the robot's position. I also added logic to trigger tile interactions (such as picking up a chip or unlocking a door) immediately after the robot moves. This allows the player to collect items seamlessly without additional button presses, just like the AI does.

```
public void keyPressed(java.awt.event.KeyEvent e) {
    if (envPanel.isCountdownActive() || playerRobot == null)
        return;

    int keyCode = e.getKeyCode();
    Position current = humanEnv.getRobotPosition(playerRobot);
    int row = current.getRow();
    int col = current.getCol();
    int newRow = row;
    int newCol = col;

    switch (keyCode) {
        case KeyEvent.VK_W:
        case KeyEvent.VK_UP:
            newRow = row - 1;
            break;
        case KeyEvent.VK_S:
        case KeyEvent.VK_DOWN:
            newRow = row + 1;
            break;
        case KeyEvent.VK_A:
        case KeyEvent.VK_LEFT:
            newCol = col - 1;
            break;
        case KeyEvent.VK_D:
        case KeyEvent.VK_RIGHT:
            newCol = col + 1;
            break;
    }

    if (humanEnv.validPos(newRow, newCol, playerRobot)) {
        humanEnv.updateRobotPos(playerRobot, newRow, newCol);
        humanEnv.handleTileInteraction(playerRobot);
    }
}

public void handleTileInteraction(Robot robot) {
    Position robotPos = getRobotPosition(robot);
    TileStatus status = tiles.get(robotPos).getStatus();
    ArrayList<String> inventory = this.robotHoldings.get(robot);

    switch (status) {
        case CHIP:
            removeFromEnvironment(status, robotPos);
            break;
        case KEY_BLUE:
            inventory.add("KEY_BLUE");
            removeFromEnvironment(status, robotPos);
            break;
        case KEY_GREEN:
            inventory.add("KEY_GREEN");
            removeFromEnvironment(status, robotPos);
            break;
        case KEY_RED:
            inventory.add("KEY_RED");
            removeFromEnvironment(status, robotPos);
            break;
        case KEY_YELLOW:
            inventory.add("KEY_YELLOW");
            removeFromEnvironment(status, robotPos);
            break;
        case DOOR_BLUE:
            inventory.remove("KEY_BLUE");
            removeFromEnvironment(status, robotPos);
            break;
        case DOOR_GREEN:
            inventory.remove("KEY_GREEN");
            removeFromEnvironment(status, robotPos);
            break;
    }
}
```

Another addition was the countdown system, which ensures that neither the AI nor the player begins the game prematurely. This was implemented in the EnvironmentPanel class using a Timer that displays a 3-second countdown before gameplay begins. The visual feedback is rendered directly over the tiles using Graphics2D, avoiding the need for an external panel. Once the countdown completes, the main game loop (also a timer) begins.

```

countdownTimer = new Timer(1000, new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (countdownSeconds > 0) {
            countdownSeconds--;
        } else {
            countdownTimer.stop();
            countdownActive = false;
            //starts the visualize simulation timer
            parent.startTimer();
        }
        repaint();
    }
});

countdownTimer.start();

```

```

if (countdownActive) {
    Graphics2D g2 = (Graphics2D) g;
    g2.setFont(new Font("Arial", Font.BOLD, 120));
    g2.setColor(Color.RED);
    String text = countdownSeconds > 0 ? countdownSeconds + " : " : "Go!";
    int stringWidth = g2.getFontMetrics().stringWidth(text);
    int stringHeight = g2.getFontMetrics().getHeight();

    int centerX = (getWidth() - stringWidth) / 2;
    int centerY = (getHeight() + stringHeight) / 2;

    g2.drawString(text, centerX, centerY);
}

```

For the AI planning, the robot first tries to plan a path to the nearest chip, but if none are reachable, it searches for a missing key and plans a path to retrieve it. If even that is unreachable, it looks for any key it can access. This flexibility ensures the AI doesn't get stuck behind locked doors and can dynamically update its plan when the environment changes. A key method here is `chooseBestTarget()` in the Robot class, which encapsulates the multi-layered prioritization logic for chips, keys, doors, and the portal. Those subgoals are what get painted on the GUI as part of the agent planning.

```

for (Robot robot : env.getRobots()) {
    if (!robot.isPlayerControlled()) {
        List<Position> path = robot.getCurrentPlan();
        if (path != null) {
            Graphics2D g2 = (Graphics2D) g;
            g2.setColor(new Color(200, 0, 0, 120));
            for (Position pos : path) {
                g2.fillRect(pos.getCol() * TILESIZE, pos.getRow() * TILESIZE, TILESIZE - 1, TILESIZE - 1);
            }
        }
    }
}

```

Finally, I added several user interface improvements to enhance the simulation's feel. This included a "Winner" label added directly between the chips remaining and inventory UI components, which displays the winning agent when the goal is met. I also added buttons for start, stop, reset, and step functionality using `JButton`, as well as a difficulty level selector (`JComboBox`) that adjusts the AI's speed by scaling the timer's delay. A screenshot of the button setup and the difficulty dropdown configuration in `VisualizeSimulation` would nicely demonstrate this final polish.

```

final int baseDelay = DELAY;

JLabel diffLabel = new JLabel("Difficulty:");
String[] levels = { "Easy", "Medium", "Hard" };
JComboBox<String> diffBox = new JComboBox<>(levels);
//default to medium
diffBox.setSelectedItem("Medium");

diffBox.addActionListener(ev -> {
    String level = (String) diffBox.getSelectedItem();
    int scaledDelay;
    switch (level) {
        case "Easy":
            scaledDelay = baseDelay * 2;
            break;
        case "Hard":
            scaledDelay = baseDelay / 2;
            break;
        default:
            scaledDelay = baseDelay;
    }
    timer.setDelay(scaledDelay);
});

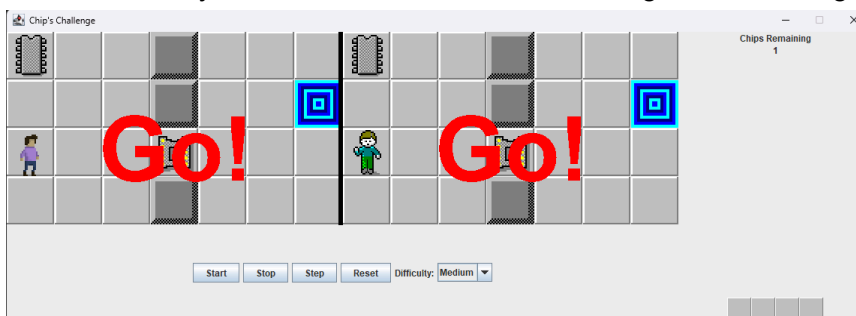
```

Together, these enhancements significantly elevate the experience beyond a standard visualization, turning it into a competitive, playable simulation that reflects both strategic AI design and player engagement.

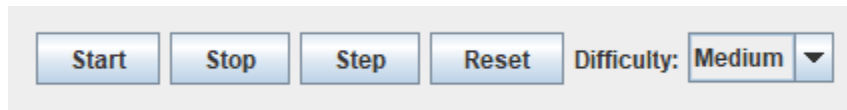
Evaluation and Results:

The outcome of my +X implementation transformed the simulation from a passive, demonstration into an interactive, competitive experience with meaningful visual and functional upgrades. The most immediate result was the successful synchronization of two separate environments, one for a human-controlled player and one for the AI planner, allowing them to operate independently but be compared directly on-screen. This setup created a dynamic space for evaluating planning algorithms, while also letting the player participate actively in the problem solving process.

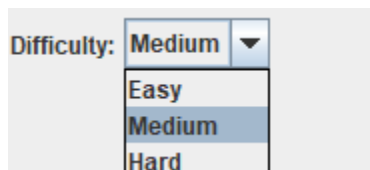
The AI robot, using A* search and subgoal planning, demonstrated robust behavior across all public maps. It was able to collect chips, unlock doors, and adapt its path based on newly accessible tiles, all without manual intervention. The player, on the other hand, provided a direct method to verify the AI's performance by offering a human benchmark. In multiple test runs, I was able to either beat or lose to the AI depending on how efficiently I moved, which validated that the difficulty level was balanced and the AI logic was working as intended.



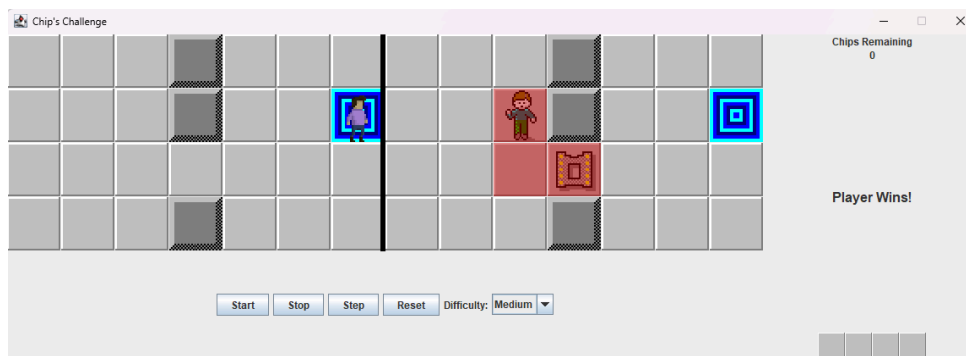
To better understand how the new features impacted usability, I ran the simulation with and without the enhancements across several scenarios. Without the +X features, the AI executed its plan silently while the user could only observe, making debugging harder and interaction nonexistent. With the enhancements, not only was the game loop more transparent (thanks to the countdown and step controls), but the planning logic could be interactively explored and tested. For example, using the “Step” button, I was able to advance the AI one decision at a time and observe how it recalculated subgoals.



The addition of the difficulty dropdown also provided a way to scale the AI’s reaction speed, making the experience either more casual or more challenging. For example, in “Hard” mode, the AI moves twice as fast, increasing the pressure on the player to optimize their path and reducing the likelihood of beating the planner through random wandering. This simple addition allowed the game to be tuned to different audiences and made it easier to observe AI performance under time constraints.



The Winner display label provided an immediate and gratifying visual confirmation of simulation outcomes. Users receive an in-game update on who reached the goal first. This small UI element significantly improved feedback clarity and marked the culmination of each simulation run with a clear conclusion.



One of the most helpful additions was the red-highlighted path preview for the AI robot. This feature visually renders the currently planned route from the AI’s current position to its next subgoal, updating dynamically whenever the plan is recalculated. Not only does this add a visually appealing aspect to the simulation, but it also significantly improves interpretability for developers and players alike. It makes the robot’s intentions visible, aiding debugging, performance evaluation, and even learning. For instance, when the AI path leads to a door, the red line halts and shifts toward a key after replanning, making the agent’s logic transparent and justifiable. This visual addition gives insight into the decision making process.

Works Referenced:

1. Ghost Town Games Ltd., "*Overcooked* [Online]," Steam, 2016. Available:
<https://store.steampowered.com/app/448510/Overcooked/>